Elementary graph algorithms: solutions

22-E01 Representations and zero weights

Derivation

Each undirected edge contributes one entry at each endpoint, so9 edges give 18 entries, in addition to 6 headers. BFS initializes 6 vertices, enqueues each at most once, and scans 18 entries if all vertices are reachable. Time is O(n+m), rather than O(nm). The 6×6 matrix has 36 cells; generally its space is $\Theta(n^2)$.

Using zero for absence loses connectivity when u→v has weight 0. If that is the only edge, reading it as absent incorrectly declares v unreachable from u. Store pair(exists,weight), or a nonnumeric NIL absence marker while keeping numeric zero for a valid weight.

Conclusion and checks

There are 18 entries. List BFS takes O(n+m); matrix space is $\Theta(n^2)$. An ∞ absence marker is also valid if ∞ is not an allowed actual edge weight.

22-E02 BFS on six vertices

Derivation

Initially queue=[d],d[d]=0, with all other distances∞. Removing d discovers b,e at distance 1 with predecessor d, leaving [b,e]. Removing b discovers c with d[c]=2,π[c]=b, leaving [e,c]. Removing e finds d already discovered, leaving [c]. Removing c discovers a with d[a]=3,π[a]=c, leaving [a]. Removing a finds b,c already discovered, leaving [].

In a,b,c,d,e,f order, distances are (3,1,2,0,1,∞), and π=(c,d,b,NIL,d,NIL). Vertex f is undiscovered: f→e permits travel from f to e, not in reverse.

Edge a→b joins distances3 and 1, differing by 2. BFS requires d[b]≤d[a]+1, here1≤4. It does not require d[a]≤d[b]+1 because b→a is absent. There is a two-edge path b→c→a, not a reverse single edge.

Conclusion and checks

The removal order d,b,e,c,a has nondecreasing distances0,1,1,2,3. Every finite predecessor chain decreases distance by one. The shortest chain to a is d→b→c→a. The absolute-difference bound applies to undirected edges.

Elementary graph algorithms: solutions (continued)

22-E03 SCCs and crossing transposed components

Derivation

The first recursion follows a→b→c→d. Events are discover a1,b2,c3,d4; finish d5,c6,b7,a8. Thus a=1/8,b=2/7,c=3/6,d=4/5. Finish order d,c,b,a reverses to candidate order a,b,c,d.

Transposed lists are a:[b],b:[a],c:[b],d:[c]. Pass two from a collects {a,b}, then skips black b. From c it sees black b and collects only {c}. From d it sees black c and collects only {d}.

Starting arbitrarily at d would reach d→c→b→a, all four vertices, without establishing mutual reachability. In the original graph d has no outgoing edge and cannot return to c. Transposition preserves mutually reachable equivalence classes; it does not confine arbitrary searches to one class. Decreasing first-pass finish values and already-black boundaries prevent crossing into unprocessed classes.

Conclusion and checks

The SCCs are {a,b},{c},{d}, whose original condensation has decreasing maximum finish values 8>6>5. An arbitrary G^T search can combine all three classes, demonstrating why root order is necessary.

22-E04 DFS edge types on the same graph

Derivation

Events are discover a1,b2,c3; finish c4,b5,a6; discover d7,e8; finish e9,d10; discover f11; finish f12. Roots a,d,f have predecessor NIL; π[b]=a,π[c]=b,π[e]=d.

Scan order gives a→b(T),b→c(T),c→a(B, gray a); after returning to a, a→c(F, black c with tin[a]<tin[c]); d→b(C, finished nonancestor b); d→e(T); e→d(B, gray d); f→e(C, black e with a disjoint interval).

Edge a→c did not discover c, so it is not tree. Interval [3,4] lies strictly inside a's [1,6], so c is a proper descendant and the edge is not cross. Tree and forward edges share the strict nesting inequality; distinguish them using π[v]=u or the recorded discovery edge.

Conclusion and checks

The 8 edges comprise3T,2B,1F,2C. Timestamps1..12 occur once, and roots are a,d,f. Descendant intervals do not establish direct parenthood, as a→c illustrates.

Elementary graph algorithms: solutions (continued)

22-E05 Valid topological orders and remaining vertices

Derivation

Orders a,b,c,d and b,a,c,d place both a,b before c, and c before d, so both are valid. Order a,c,b,d reverses b→c and is invalid; correctly ordering c→d cannot repair another reversed edge.

Initially indegree=(0,0,2,1),queue=[a,b]. Output a makes c's indegree1,queue=[b]. Output b makes it0,queue=[c]. Output c makes d's indegree0,queue=[d]. Output d empties the queue, with all four vertices output.

After adding d→b,d→z, indegrees are a0,b1,c2,d1,z1. The initial queue=[a]. Output a changes c from 2 to 1 and empties the queue. The remaining vertices are {b,c,d,z}; b→c→d→b is a cycle. Vertex z is downstream from d and has no outgoing edge, so it lies on no cycle.

Conclusion and checks

Kahn correctly reports a cycle after outputting only a. Every remaining vertex has a remaining incoming edge, and following incoming edges finds some cycle. It need not find a cycle containing the starting vertex.

22-E06 White paths versus ordinary reachability

Derivation

Events are discover a1,b2; finish b3; discover c4; finish c5,a6. Just before c's discovery, a is gray, b black, and c white. The white set is {c}. Path c→b is not all white.

Intervals [2,3] and [4,5] are disjoint; π[b]=π[c]=a, so b,c are siblings. Scanning c→b does not recurse, and an old discovery cannot move into a new subtree. The theorem examines the entire path at the single instant just before the root u is discovered; ordinary existence of a graph path is insufficient.

Conclusion and checks

Reachability of b from c does not make b a DFS descendant of c. The zero-length path from c to itself is white, matching the convention that c is its own descendant. A proper-descendant statement must exclude v=u.

Elementary graph algorithms: solutions (continued)

22-E07 Shortest-path counts and bit cost

Derivation

Discover 1,2 from 0 at distance 1 with count 1 each. Processing1 gives 3,4 distance 2 and adds 1 to each count. Processing2 again satisfies the distance equality for both discovered vertices and adds 1, giving ways[3]=ways[4]=2. Processing3 first discovers 5 at distance 3 and adds 2. Processing4 adds another2, yielding ways[5]=4.

Thus d=(0,1,1,2,2,3),ways=(1,1,1,2,2,4). Counting only on first discovery omits later predecessors at the same shortest distance. Here it would already give incorrect counts 1 for 3,4, and ultimately1 for 5.

With L two-vertex layers, a single source connected to the first layer, complete connections between consecutive layers, and a single sink after the last layer, there are 2^L shortest source-to-sink paths. Since n=2L+2, their number is 2^Θ(n) and binary length Θ(n). The algorithm performs O(n+m) integer additions. In a bit model each can cost O(n), giving a direct upper bound O(n(n+m)) bit operations; linear addition count does not eliminate operand length.

Conclusion and checks

Ordering within a layer does not affect the final count because every shortest predecessor edge contributes once and the preceding layer finishes before the next is dequeued. Linear time assumes unit-cost arithmetic or sufficiently large fixed-width integers. Modular counting must state its modulus.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Elementary graph algorithms: solutions (continued)

Verification code appendix: breadth_first_search.py

```
"""BFS distances on the course's layered example."""

from collections import deque

def bfs(graph, source):
    distance = {source: 0}
    queue = deque([source])
    while queue:
        u = queue.popleft()
        for v in graph.get(u, ()):
            if v not in distance:
                distance[v] = distance[u] + 1
                queue.append(v)
    return distance


graph = {"s": ["a", "b"], "a": ["c"], "b": ["c"], "c": []}
assert bfs(graph, "s") == {"s": 0, "a": 1, "b": 1, "c": 2}
print(bfs(graph, "s"))
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Elementary graph algorithms: solutions (continued)

Verification code appendix: graph_search_checks.py

```
"""Graph searches checked against independent all-pairs path recurrences."""
from collections import deque
from itertools import product
from math import inf

SAMPLE={'a':['b','c'],'b':['c'],'c':['a'],'d':['b','e'],'e':['d'],'f':['e']}


def transpose(graph):
    result={u:[] for u in graph}
    for u,neighbors in graph.items():
        for v in neighbors:result[v].append(u)
    return result


def bfs(graph,source):
    distance={u:inf for u in graph};parent={u:None for u in graph}
    ways={u:0 for u in graph};distance[source]=0;ways[source]=1
    queue=deque([source]);trace=[]
    while queue:
        u=queue.popleft()
        for v in graph[u]:
            if distance[v]==inf:
                distance[v]=distance[u]+1;parent[v]=u;queue.append(v)
            if distance[v]==distance[u]+1:ways[v]+=ways[u]
        trace.append((u,tuple(queue)))
    return distance,parent,ways,trace


def dfs(graph,order=None):
    colors={u:0 for u in graph};tin={};tout={};parents={u:None for u in graph}
    edges=[];finished=[];trees=[];white_reachable={};time=0
    def visit(u,tree):
        nonlocal time
        # Snapshot before u changes from white; independent reachability search.
        white={v for v in graph if colors[v]==0};seen={u};pending=[u]
        while pending:
            x=pending.pop()
            for y in graph[x]:
                if y in white and y not in seen:seen.add(y);pending.append(y)
        white_reachable[u]=seen
        time+=1;tin[u]=time;colors[u]=1;tree.append(u)
        for v in graph[u]:
            if colors[v]==0:
                edges.append((u,v,'T'));parents[v]=u;visit(v,tree)
            elif colors[v]==1:edges.append((u,v,'B'))
```

Elementary graph algorithms: solutions (continued)

```
            else:edges.append((u,v,'F' if tin[u]<tin[v] else 'C'))
        colors[u]=2;time+=1;tout[u]=time;finished.append(u)
    for u in graph if order is None else order:
        if colors[u]==0:
            tree=[];visit(u,tree);trees.append(tree)
    return dict(tin=tin,tout=tout,parent=parents,edges=edges,
                finished=finished,trees=trees,white=white_reachable)


def scc(graph,first=None):
    first=dfs(graph) if first is None else first
    return dfs(transpose(graph),reversed(first['finished']))['trees']


def kahn(graph):
    incoming={u:0 for u in graph}
    for neighbors in graph.values():
        for v in neighbors:incoming[v]+=1
    queue=deque(u for u in graph if incoming[u]==0);order=[]
    while queue:
        u=queue.popleft();order.append(u)
        for v in graph[u]:
            incoming[v]-=1
            if incoming[v]==0:queue.append(v)
    return order if len(order)==len(graph) else None


def all_pairs(graph):
    # Floyd-style path recurrence, including self-loop cycle detection separately.
    vertices=list(graph);distance={(u,v):0 if u==v else inf for u in vertices for v in
vertices}
    reach={(u,v):v in graph[u] for u in vertices for v in vertices}
    for u in vertices:
        for v in graph[u]:distance[u,v]=min(distance[u,v],1)
    for k in vertices:
        for u in vertices:
            for v in vertices:
                distance[u,v]=min(distance[u,v],distance[u,k]+distance[k,v])
                reach[u,v]=reach[u,v] or (reach[u,k] and reach[k,v])
    return distance,reach


def verify_graph(graph):
    vertices=list(graph);distance,reach=all_pairs(graph);first=dfs(graph)
    tin,tout=first['tin'],first['tout']
    assert sorted([*tin.values(),*tout.values()])==list(range(1,2*len(graph)+1))
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Elementary graph algorithms: solutions (continued)

```
    assert {u:frozenset(v for v in vertices if tin[u]<=tin[v] and tout[v]<=tout[u])
            for u in vertices}=={u:frozenset(v) for u,v in first['white'].items()}
    for u in vertices:
        for v in vertices:
            if u==v:continue
            assert (tout[u]<tin[v] or tout[v]<tin[u]
                    or tin[u]<tin[v]<tout[v]<tout[u]
                    or tin[v]<tin[u]<tout[u]<tout[v])
    for u,v,kind in first['edges']:
        if kind=='T':assert first['parent'][v]==u and tin[u]<tin[v]<tout[v]<tout[u]
        elif kind=='F':assert first['parent'][v]!=u and tin[u]<tin[v]<tout[v]<tout[u]
        elif kind=='B':assert u==v or tin[v]<tin[u]<tout[u]<tout[v]
        else:assert tout[v]<tin[u]
    cyclic=any(reach[u,u] for u in vertices)
    assert cyclic==any(kind=='B' for _,_,kind in first['edges'])
    order=kahn(graph);assert (order is None)==cyclic
    if not cyclic:
        for candidate in (order,list(reversed(first['finished']))):
            positions={u:i for i,u in enumerate(candidate)}
            assert all(positions[u]<positions[v] for u in vertices for v in graph[u])
    groups=scc(graph,first);membership={u:i for i,group in enumerate(groups) for u in group}
    assert len(membership)==len(graph)
    for u in vertices:
        for v in vertices:
            assert (membership[u]==membership[v])==(u==v or (reach[u,v] and reach[v,u]))
    finishmax={i:max(tout[u] for u in group) for i,group in enumerate(groups)}
    for u in vertices:
        for v in graph[u]:
            if membership[u]!=membership[v]:
                assert finishmax[membership[u]]>finishmax[membership[v]]
    for source in vertices:
        computed,parent,ways,trace=bfs(graph,source)
        for v in vertices:
            assert computed[v]==distance[source,v]
            if v!=source and computed[v]<inf:
                assert v in graph[parent[v]] and computed[parent[v]]+1==computed[v]
            # Enumerate simple paths independently; shortest unit-edge paths are simple.
            count=0
            def paths(u,seen,length):
                nonlocal count
                if length>distance[source,v]:return
                if u==v:
                    if length==distance[source,v]:count+=1
                    return
                for w in graph[u]:
                    if w not in seen:paths(w,seen|{w},length+1)
```

Elementary graph algorithms: solutions (continued)

```
            paths(source,{source},0)
            assert ways[v]==count
        for _,queue in trace:
            values=[computed[u] for u in queue]
            assert values==sorted(values)
            assert not values or values[-1]<=values[0]+1


def main():
    total=0;sources=0
    for n in range(5):
        for mask in range(1<<(n*n)):
            graph={u:[v for v in range(n) if mask>>(n*u+v)&1] for u in range(n)}
            verify_graph(graph);total+=1;sources+=n
    first=dfs(SAMPLE)
    assert first['tin']==dict(a=1,b=2,c=3,d=7,e=8,f=11)
    assert first['tout']==dict(a=6,b=5,c=4,d=10,e=9,f=12)
    assert first['edges']==[('a','b','T'),('b','c','T'),('c','a','B'),('a','c','F'),
                            ('d','b','C'),('d','e','T'),('e','d','B'),('f','e','C')]
    assert scc(SAMPLE)==[['f'],['d','e'],['a','c','b']]
    d,p,_,trace=bfs(SAMPLE,'d')
    assert d==dict(a=3,b=1,c=2,d=0,e=1,f=inf)
    assert trace==[('d',('b','e')),('b',('e','c')),('e',('c',)),('c',('a',)),('a',())]
    diamond={0:[1,2],1:[3,4],2:[3,4],3:[5],4:[5],5:[]}
    assert list(bfs(diamond,0)[2].values())==[1,1,1,2,2,4]
    assert kahn({'a':['c'],'b':['c'],'c':['d'],'d':[]})==['a','b','c','d']
    assert kahn({'a':['c'],'b':['c'],'c':['d'],'d':['b']}) is None
    # The wrong root order really merges distinct SCCs in the transpose.
    tiny={'x':['y'],'y':[]}
    assert dfs(transpose(tiny),['y','x'])['trees']==[['y','x']]
    assert scc(tiny)==[['x'],['y']]
    # A finished reachable vertex need not be a descendant.
    wrong=dfs({'a':['b','c'],'b':[],'c':['b']})
    assert wrong['tout']['b']<wrong['tin']['c'] and 'b' not in wrong['white']['c']
    # Reversing every adjacency order leaves all invariant checks intact.
    verify_graph({u:list(reversed(v)) for u,v in SAMPLE.items()})
    print(f'PASS directed_graphs_with_self_loops_n0_to4={total} BFS_sources={sources} '
          'distance=all_pairs_oracle shortest_counts=simple_path_oracle '
          'SCC=mutual_reachability DFS=white_paths_and_intervals topo=cycle_equivalence '
          'sample_all8_edge_classes_and_two_pass_trace=True')


if __name__=='__main__':main()
```

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study